Piano di sviluppo — Web App di Georeferenziazione Mappe Storiche

> **Obiettivo:** Applicazione web a pagina singola (`index.html`), ospitata su GitHub Pages, che permette di sovrapporre una mappa storica su una mappa moderna, georeferenziarla tramite Ground Control Points (GCP) e salvare il risultato in formato **KML** o **GeoTIFF**.

Indice

1. Stack tecnologico
2. Struttura del repository
3. Architettura dell'applicazione
4. Fase 1 — Setup GitHub Pages
5. Fase 2 — Interfaccia utente
6. Fase 3 — Caricamento immagine storica
7. Fase 4 — Overlay e posizionamento
8. Fase 5 — Gestione GCP
9. Fase 6 — Export KML
10. Fase 7 — Export GeoTIFF (client-side)
11. Fase 8 — Persistenza locale
12. Fase 9 — Selezione mappa di base
13. Fase 10 — Deploy su GitHub Pages
14. Limiti tecnici e alternative
15. Dipendenze esterne

16. Roadmap prioritizzata

---

1. Stack tecnologico

Componente	Libreria / API	Note
Mappa base	**Leaflet.js 1.9**	Leggero, CDN
Overlay immagine	**Leaflet.DistortableImage**	Warp affine con maniglie
Export KML	Generazione XML nativa JS	Nessuna dipendenza
Export GeoTIFF	**geotiff.js** + trasformata affine	Client-side, puro JS
Proiezione coordinate	**proj4.js**	Conversione WGS84 ↔ CRS locali
File locale	FileReader API	Nativo nel browser
Hosting	**GitHub Pages**	Gratuito, zero backend
Persistenza	localStorage + JSON export	Nessun server

---

2. Struttura del repository

```
nomeRepo/
├── index.html          # Unico file — app completa
├── README.md           # Istruzioni utente
```

```
└── examples/
    └── sample-map.jpg  # Mappa di esempio per test
```

+ add code replace

> Tutto il codice (HTML, CSS, JS) è contenuto in `index.html`. Le librerie vengono caricate da CDN. Nessun processo di build.

---

3. Architettura dell'applicazione

```
┌──────────────────────────────────────────────────────┐
│                      index.html                      │
│                                                      │
│  ┌──────────────────────────┐  ┌──────────────────┐  │
│  │      Mappa Leaflet       │  │  Pannello laterale │ │
│  │                          │  │                  │ │
│  │  [Mappa base OSM/Esri]   │  │ 📁 Carica immagine │ │
│  │                          │  │ 🔲 Lista GCP     │ │
│  │  [Overlay immagine       │  │ 👁 Opacità slider │ │
│  │   storica distorcibile]  │  │ 💾 Export KML    │ │
│  │                          │  │ 🗺 Export GeoTIFF │ │
│  │  [Marker GCP]            │  │ 📦 Salva progetto │ │
│  └──────────────────────────┘  └──────────────────┘  │
│                                                      │
│  Moduli JS interni:                                  │
│  ├── ImageLoader       (FileReader API)              │
│  ├── OverlayController (Leaflet.DistortableImage)    │
│  ├── GcpManager        (click handler + tabella)     │
│  ├── AffineTransform   (calcolo trasformata)         │
│  ├── KmlExporter       (generazione XML)             │
```

```
│  ├── GeoTiffExporter   (geotiff.js + canvas)          │
│  └── ProjectStore      (localStorage + JSON)          │
└──────────────────────────────────────────────────────┘
```

+ add code replace

---

4. Fase 1 — Setup GitHub Pages

Operazioni

1. Creare un repository pubblico su GitHub (es. `georef-storico`)
2. Caricare `index.html` nella root del branch `main`
3. Andare in **Settings → Pages → Source → Deploy from branch → main → / (root)**
4. Dopo 1-2 minuti l'app è disponibile su: `https://username.github.io/georef-storico/`

Aggiornamenti futuri

Ogni `git push` su `main` rideploya automaticamente l'app in 30-60 secondi.

---

5. Fase 2 — Interfaccia utente

Layout

- **Sinistra (80%):** mappa Leaflet full-height
- **Destra (20%):** pannello verticale scorrevole con tutti i controlli

Toolbar superiore

```
[ Carica mappa ] [ 🖐 Pan ] [ 📍 Aggiungi GCP ] [ 🗑 Rimuovi GCP ] [ Opacità: ▒▒▒▒▒▒▒▒▒▒ 70% ]
```

+ add code replace

Pannello GCP

Tabella live con:

- `#` — numero progressivo
- `Pixel X / Y` — coordinate sull'immagine storica
- `Lon / Lat` — coordinate geografiche reali
- `×` — pulsante rimozione

Sezione Export

```
[ 💾 Scarica KML ]  [ 🗺 Scarica GeoTIFF ]  [ 📦 Esporta progetto JSON ]
```

+ add code replace

Shortcut tastiera

Tasto	Azione
`G`	Attiva modalità aggiungi GCP
`Esc`	Modalità pan
`Delete`	Rimuovi GCP selezionato

Tasto	Azione
`Ctrl+S`	Salva progetto in localStorage

---

6. Fase 3 — Caricamento immagine storica

Metodo

- Drag & drop sull'area mappa **oppure** click su "Carica mappa"
- API `FileReader` legge il file come `dataURL` (base64)
- L'immagine viene passata a `Leaflet.DistortableImage`

Formati supportati

`JPG`, `PNG`, `WEBP`, `BMP` — qualsiasi formato supportato dal tag `<img>` del browser.

Limite pratico

File fino a ~100 MB funzionano nei browser moderni. Oltre, il GeoTIFF export diventa lento. Si consiglia di ridimensionare immagini superiori a 8000×8000 px prima del caricamento.

---

7. Fase 4 — Overlay e posizionamento

Libreria: `Leaflet.DistortableImage`

Permette di:

- **Traslare** l'immagine (drag)
- **Ruotare** (maniglia angolare)

- **Scalare** (maniglie laterali)
- **Distorcere** gli angoli singolarmente (warp affine)

Workflow utente

1. L'immagine storica appare centrata sulla vista corrente
2. L'utente la posiziona approssimativamente sull'area corretta
3. Regola scala e rotazione visivamente
4. Poi passa alla georeferenziazione puntuale con i GCP

Slider opacità

Valore 0–100%, applicato in tempo reale al layer overlay.
Utile per confrontare i dettagli tra mappa storica e mappa moderna sottostante.

---

8. Fase 5 — Gestione GCP

I **Ground Control Points** sono la chiave della georeferenziazione precisa. Ogni GCP associa un punto dell'immagine storica a una coordinata geografica reale.

Aggiunta di un GCP

1. L'utente attiva la modalità GCP (tasto `G` o toolbar)
2. **Click sull'immagine storica** → registra le coordinate pixel `(px, py)`
3. **Click sulla mappa moderna** → registra `(lon, lat)` reali
4. Il GCP appare nella tabella e come marker sulla mappa

Numero minimo di GCP

GCP	Trasformata possibile
3	Affine (traslazione + rotazione + scala)
4+	Proiettiva / bilineare
6+	Polinomiale di 2° ordine (warp preciso)

> Con meno di 3 GCP l'export è disabilitato.

Calcolo errore (RMSE)

Dopo ogni GCP aggiunto, l'app calcola e mostra il **Root Mean Square Error** residuo in metri. Valori accettabili dipendono dalla scala della mappa storica.

---

9. Fase 6 — Export KML

Cos'è il KML

Formato XML di Google per dati geografici, supportato da Google Earth, Google Maps, QGIS, ArcGIS.

Struttura generata

```xml
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
  <GroundOverlay>
    <name>Mappa storica georeferenziata</name>
    <Icon>
      <href>...</href>
    </Icon>
    <LatLonBox>
      <north>45.4125</north>
      <south>45.3980</south>
      <east>11.8890</east>
      <west>11.8720</west>
      <rotation>2.35</rotation>
    </LatLonBox>
  </GroundOverlay>
</kml>
```

+ add code replace

Note implementative

- L'immagine storica viene **embedded** nel KML come base64, quindi il file è autocontenuto
- `LatLonBox` viene calcolato dai 4 angoli dell'overlay dopo la trasformata affine
- La `<rotation>` compensa la rotazione applicata dall'utente
- Il file KML generato viene scaricato via `Blob` + `URL.createObjectURL`

Compatibilità

Software	Apertura diretta
Google Earth Pro	✅
Google Maps (import)	✅
QGIS	✅
ArcGIS Pro	✅
Leaflet (plugin)	✅

---

10. Fase 7 — Export GeoTIFF (client-side)

Il GeoTIFF è il formato raster georeferenziato standard in ambito GIS. Include i metadati di proiezione e trasformata affine direttamente nell'header del file.

Approccio tecnico

Step 1 — Disegno su Canvas

```
Immagine originale → <canvas> HTML → pixel buffer RGBA
```

+ add code replace

Step 2 — Calcolo trasformata affine

Con i GCP raccolti si calcola la **trasformata affine 2D** che mappa pixel → coordinate geografiche:

```
| X_geo |    | a  b  c | | px |
| Y_geo | = | d  e  f | | py |
| 1     |    | 0  0  1 | | 1  |
```

+ add code replace

I 6 parametri `(a, b, c, d, e, f)` vengono calcolati con **least squares** sui GCP disponibili.

Step 3 — Scrittura GeoTIFF

La libreria **geotiff.js** scrive:

- I pixel dell'immagine (da Canvas)
- Il **ModelTiepointTag** (origine geografica)
- Il **ModelPixelScaleTag** (risoluzione in gradi/metro per pixel)
- Il **ProjectedCSTypeGeoKey** (sistema di riferimento, default WGS84 / EPSG:4326)

```javascript
```

```
// Pseudocodice semplificato
const tiff = GeoTIFF.writeArrayBuffer({
  width: imgWidth,
  height: imgHeight,
  data: pixelBuffer,        // Uint8Array RGBA
  metadata: {
    ModelTiepointTag: [0, 0, 0, lon_origine, lat_origine, 0],
    ModelPixelScaleTag: [gradi_per_pixel_x, gradi_per_pixel_y, 0],
    GTModelTypeGeoKey: 2,   // Geographic
    GeographicTypeGeoKey: 4326  // WGS84
  }
});
```

+ add code replace

Step 4 — Download

javascript

```
const blob = new Blob([tiff], { type: 'image/tiff' });
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = 'mappa_georeferenziata.tif';
a.click();
```

+ add code replace

Compatibilità GeoTIFF output

Software	Apertura diretta
QGIS	✅
ArcGIS Pro	✅
GDAL / gdalinfo	✅
Google Earth Pro	✅
MapWarper	✅ (import)

Limitazione importante

La trasformata calcolata in JS è **affine** (3 GCP, 6 parametri). Per distorsioni ottiche complesse delle mappe storiche (proiezioni storiche non standard) serve un warp **polinomiale** di ordine superiore, che richiede GDAL o un backend. In tal caso l'app esporta i GCP e l'utente completa il processo in QGIS.

---

11. Fase 8 — Persistenza locale

Salvataggio automatico

Ad ogni modifica (aggiunta GCP, spostamento overlay) il progetto viene serializzato in `localStorage`:

```
json
```

```
{
  "imageDataUrl": "...",
  "overlayBounds": [[45.40, 11.87], [45.41, 11.89]],
  "rotation": 2.35,
  "opacity": 0.7,
  "gcps": [
    { "px": 120, "py": 340, "lon": 11.878, "lat": 45.401 }
  ]
}
```

+ add code replace

Export / Import progetto

- **Esporta progetto:** scarica il JSON completo (include l'immagine in base64)
- **Importa progetto:** drag & drop del file JSON → ripristino completo della sessione

> ⚠️ `localStorage` ha un limite di ~5-10 MB. Per immagini grandi usare l'export JSON su file.

---

12. Fase 9 — Selezione mappa di base

Selettore tile layer nell'angolo superiore della mappa:

Layer	Utilità
OpenStreetMap Standard	Riferimento generale
Esri World Imagery	Foto satellitare — utile per edifici e strade
Stamen Toner	Bianco/nero — riduce rumore visivo
CartoDB Positron	Minimal chiaro — massima leggibilità overlay

---

13. Fase 10 — Deploy su GitHub Pages

Prima pubblicazione

```bash
git init
git add index.html README.md
git commit -m "Prima versione app georeferenziazione"
git remote add origin https://github.com/username/georef-storico.git
git push -u origin main
```

+ add code replace

Poi: **Settings → Pages → Deploy from branch → main → / (root) → Save**

Aggiornamenti

```bash
```

```
git add index.html
git commit -m "Aggiunta funzione export GeoTIFF"
git push
```

+ add code replace

L'app si aggiorna in automatico entro 1 minuto.

URL finale

```
https://username.github.io/georef-storico/
```

+ add code replace

---

14. Limiti tecnici e alternative

Warp polinomiale avanzato

Il browser non può eseguire GDAL. Per distorsioni complesse (mappe storiche con proiezioni non euclidee):

Opzione A — Export GCP → QGIS L'app esporta i GCP come file `.points` compatibile con QGIS Georeferencer. L'utente apre QGIS e completa il warp polinomiale localmente.

Opzione B — Integrazione MapWarper L'app esporta KML/GeoTIFF abbozzato. L'utente lo importa su MapWarper per un warp cloud-based gratuito.

Opzione C — Backend leggero (futuro) Aggiungere un endpoint serverless (Vercel Functions / Cloudflare Workers) che riceve immagine + GCP e restituisce GeoTIFF warpato con GDAL. Mantiene il frontend statico su GitHub Pages.

Dimensione file

File GeoTIFF client-side non compressi possono essere grandi. Consigliato comprimere l'immagine in ingresso (max 4000×4000 px) per export fluido.

---

15. Dipendenze esterne

Tutte caricate da CDN — nessuna installazione locale richiesta.

```html
<!-- Leaflet -->
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

<!-- Leaflet.DistortableImage -->
<script src="https://unpkg.com/leaflet-distortableimage@0.8.0/dist/leaflet.dist

<!-- geotiff.js (export GeoTIFF) -->
<script src="https://cdn.jsdelivr.net/npm/geotiff@2.1.3/dist-browser/geotiff.js

<!-- proj4.js (proiezioni) -->
<script src="https://cdn.jsdelivr.net/npm/proj4@2.11.0/dist/proj4.js"></script>
```

+ add code replace

---

16. Roadmap prioritizzata

MVP — Versione 1.0

- [x] Setup GitHub Pages
- [x] Mappa Leaflet con layer switcher
- [x] Caricamento immagine da file locale
- [x] Overlay con `Leaflet.DistortableImage`
- [x] Slider opacità
- [x] Gestione GCP (min. 3 punti)
- [x] Export **KML** con immagine embedded
- [x] Export **GeoTIFF** affine con geotiff.js
- [x] Salvataggio sessione in localStorage

Versione 1.1

- [ ] Calcolo e visualizzazione RMSE per GCP
- [ ] Import progetto da JSON
- [ ] Export GCP compatibile QGIS (`.points`)
- [ ] Modalità confronto: split screen storico/moderno

Versione 1.2

- [ ] Supporto proiezioni storiche (proj4.js + EPSG registry)
- [ ] Warp polinomiale di 2° ordine client-side
- [ ] Annotazioni vettoriali sull'overlay (poligoni, etichette)
- [ ] Export GeoJSON degli annotation layer

Piano redatto per sviluppo single-file HTML — zero dipendenze server — hosting gratuito GitHub Pages.